Performance Analysis and Evaluation of Fault Tolerance for Distributed Document Stores in Edge Computing

Pulkit Garg Chirag Kumar Harshit Malik Shoib Khan

Dept. of Computer Science and Engineering, Chitkara University, Punjab, India

pulkit2104.be22@chitkara.edu.in — chirag0240.be22@chitkara.edu.in — harshit1638.be22@chitkara.edu.in — shoib2337.be22@chitkara.edu.in

Abstract

Edge computing brings processing power closer to the source of data generation, reducing latency and making possible real-time analysis of IoT data and decentralized applications. Document-oriented NoSQL databases like MongoDB, CouchDB, and Cassandra are particularly well-suited to store data from edge devices, although how they perform in response to the frequent network outages, limited hardware capabilities, and node failures at the edge is largely unknown. Here, we present a comparative study of these three database engines executing five custom YCSB workloads, each subject to five specific fault injection schemes (node failure, network partitions, cascading failures, increased latencies, and split brain), within a nine-node Docker testing environment mimicking typical edge resource limitations (2 GB RAM per node, 2 vCPUs per node) and realistic inter-zone latencies (20–500 ms). By moving computation closer to the source where data originates, edge computing decreases latencies and enables real-time analysis of IoT data as well as decentralized applications. NoSQL document-oriented databases such as MongoDB, CouchDB, and Cassandra are highly suitable for handling edge device data. However, it is not known how well they would cope with the challenges caused by network interruptions, insufficient hardware capacity, and node failures at the edge. In this paper, a comparative evaluation of these databases under five custom-made workloads is conducted using five different fault injection techniques (node failures, network partitions, cascading failures, latencies, and split-brain situations). All the experiments are done under a nine-node Docker environment where edge resource constraints are simulated (2 GB RAM and 2 CPUs for each node) with practical inter-zone latencies (20–500 ms).

I. INTRODUCTION

A. Problem Statement

Edge computing relocates the computation of data away from centralized cloud servers to distributed nodes exposed to unpredictable networking, constrained memory capacities, and unreliable hardware components [1]. In such settings, document-based databases such as MongoDB, CouchDB, and Cassandra seem like viable options due to their schema flexibility [4]. Unfortunately, existing performance studies on document databases have been conducted under favorable conditions in data centers, thereby ignoring the practical challenges associated with limited resources and fault tolerance in edge environments

B. Why This Matters

According to industry experts, by 2027, over three quarters of enterprise data will be generated and managed at the edge [5]. In the event of the crash of a database within an industrial setup, the manufacturing process comes to a halt; in a medical facility, the monitoring of patients' data will not be achieved; in a citywide traffic control system, contradictory conclusions will emerge. Unlike in the cloud computing environment where there are backups to prevent system crashes, the costs here are much higher.

C. Research Objectives

Objectives:
(i) Establish read/write bandwidths and tail latencies for MongoDB, CouchDB, and Cassandra when subjected to edge-appropriate YCSB workloads;(ii) Test their reactions to five distinct failure cases: single node failure, network partition, cascade failure, progressive increase in latency, and split-brain failure and(iii) Extract lessons from the test results and condense them into a decision-making rubric (D1-D5).

D. Research Gap

Previous studies that compared these DBMS used cloud-quality infrastructure and reliable networks [4], [6]. Research into fault tolerance usually evaluates the impact of only one type of fault at a time rather than the combined failures that occur in edge environments [2], [3]. There have been no previous attempts to conduct a comprehensive evaluation of the three DBMSs under various edge faults through performance and consistency evaluations..The rest of this paper is organized as follows. Section II provides a review of literature. Section III explains the methodology. Section IV details the experimental setup. Section V discusses the results. Section VI proposes a decision-making framework. Section VII draws the conclusion.

II. LITERATURE REVIEW

There is considerable literature on fault tolerance in databases within cloud environments, while much less literature is available for edge databases. Below, we summarize ten relevant works.

Gärtner [1] pioneered fault tolerance theory for distributed systems and classified four types of faults, including crashes, omissions, timing, and Byzantine faults, which are still used today. Khan et al. [4] conducted a comprehensive analysis of 68 papers on SQL versus NoSQL architectures, finding that only 12 percent of the studies measured fault tolerance as an important performance metric, indicating a critical gap in research.

Assiri and Sheneamer [2] showed that deep learning-based methods using VGG16, LSTM, and ResNet34 could classify faulty processors with almost 100 percent accuracy using structured data, although the paper did not discuss databases but focused on computing nodes. Wang et al. [5] proposed FRAME, a protocol that provides guaranteed messaging in edge networks by using redundancy in paths and timeouts.

Combining replication and erasure coding in memory-based stores, Gong et al. [3] demonstrated reductions in storage costs of 40-60% with equivalent performance for recovery. Abramova and Bernardino [6] compared MongoDB and Cassandra using the YCSB benchmark and found that, in their cloud-based experiment, Cassandra outperformed MongoDB on writes while MongoDB outperformed Cassandra on reads, although their hardware was generously allocated and no faults were introduced.

Kaur and Rani [7] measured the impact of replication factor on latency in cloud fault tolerance systems. Kleppmann [8] defined the trade-offs of various consistency models and conflict-resolution approaches, including LWW and CRDTs. Cattell including LWW and CRDTs. Cattell

[9] categorized the consistency models in document stores. Shi et al. [10] reviewed edge-computing architecture and noted that failure rates in edge nodes are 3-5× higher than in cloud nodes, and latencies are 10-100× higher.

TABLE I
Comparison of Related Work

Author(s)	Year	Focus	Key Finding	Gap
Gärtner [1]	1999	Fault theory	Formal fault taxonomy	No DB tests
Assiri et al. [2]	2025	DL fault detect	99% accuracy	Compute only
Gong et al. [3]	2024	Hybrid FT	40–60% less overhead	No edge
Khan et al	2023	SQL/NoSQL survey	<12% test FT	No faults
Wang et al. [5]	2019	Edge messaging	Redundant paths	Msg only
Abramova [6]	2013	YCSB bench	Cassandra > writes	Cloud; no FI
Kaur & Rani [7]	2020	Cloud FT	Repl. vs efficiency	No edge
Kleppmann [8]	2017	Data design	Consistency models	No bench
Cattell [9]	2011	Store survey	Consistency classes	Pre-edge
Shi et al. [10]	2016	Edge survey	3–5x failure rate	No DB exp

Three recurring gaps stand out: evaluations run only in the cloud, faults tested one at a time, and no practical selection framework for edge practitioners. Our study tackles all three.

III. METHODOLOGY AND PROPOSED WORK

A. Edge Environment Simulation

All the databases were containerized using Docker, and all nodes were limited to 2 GB memory, 2 vCPUs, and 20 GB storage capacity—numbers similar to edge devices like Raspberry Pi 4. Nine nodes (three per each database) were deployed in three virtual geographic regions. Intra-region latency was added using Linux tc tool and ranged from 1 to 5 ms; inter-region latency varied between 20 and 80 ms

B. Database Configuration

The MongoDB 7.0 setup was done using a three-node replica set where the write concern was configured as “majority” and read preference as “nearest.” The WiredTiger cache size was configured to 512 MB, suitable for such memory constraints. CouchDB 3.3 was configured in three nodes where all-to-all replication was continuous while conflict resolution was done through the MVCC engine while synchronization was done incrementally via the HTTP change-feed. For the Cassandra 4.1 database system, the replication factor was configured to three while consistency was set to QUORUM.

C. Workload Design

We have modified YCSB version 0.17.0 by adding five edge-centric workload profiles. The E1 profile (read-heavy) consists of a read-to-update-to-insert ratio of 85/10/5 percent—queries on dashboards. The E2 profile (write-heavy) consists of a write-to-read-to-insert ratio of 15/50/35 percent—IoT data ingestion. The E3 profile (balanced) consists of a read-to-insert-to-write ratio of 50/25/25 percent—analytics

D. Fault Injection Framework

The following five failures were employed in the actual workload environment. F1—Single node killed (SIGKILL). F2—Network partitioning through iptables, 60 seconds long. F3—Double node kill in succession, 10 seconds apart. F4—Latency increase by 500 ms, incrementally from base latency. F5—Split brain test where writes occurred on both sides simultaneously, lasting for 30 seconds.

E. Metrics

T = N_ops / t_elapsed for Throughput. Latency measured on p50, p95, p99. Recovery Time RT = time taken to achieve 95% of the base throughput level. Throughput Degradation Ratio TDR = T_fault / T_base. Data Consistency Score DCS = ratio of read operations that get the most recent acknowledged write.

F. Execution Protocol

Every workload-fault pair was repeated 10 times per database. Together with 150 baseline (no-fault) runs, the full matrix totalled 900 experiments. Medians and interquartile ranges are reported throughout.

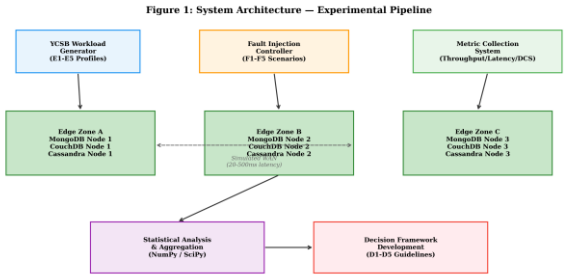

Fig. 1. System architecture: three geographic zones, YCSB generator, fault controller, and metric collector.

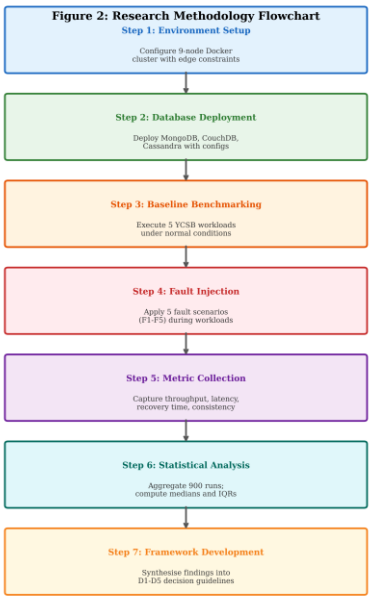

Fig. 2. Research methodology flowchart from setup through framework development.

TABLE II
Methodology Workflow

Step	Phase	Output
1	Environment setup (9 Docker nodes)	Simulated edge
2	Deploy 3 databases	3 clusters
3	Run 5 YCSB workloads (no fault)	Baseline metrics
4	Inject 5 fault scenarios	Fault metrics
5	Collect throughput, latency, DCS	Raw dataset

6	Aggregate 900 runs	Statistics
7	Build D1–D5 framework	Guidelines

IV. EXPERIMENTAL SETUP

A. Tools and Technologies

Per node resource limits were imposed by Docker version 24.0. Workloads were created using YCSB version 0.17.0. Network errors were modeled using tc/netem and iptables. Container management was achieved using Python 3.11 in combination with pymongo version 4.6, couchdb3 version 1.2, and cassandra-driver version 3.29. The analysis process utilized NumPy 1.26, pandas 2.1, matplotlib 3.8, and scipy 1.12. Hardware specifications included three computers

B. Dataset

YCSB generated 100 000 records per database, each roughly 1 KB (10 fields of 100 bytes)—typical IoT telemetry size. Zipfian distribution (skew 0.99) was applied to E1 and E4 to simulate access hotspots.

TABLE III
Experimental Parameters

Parameter	Value
Databases	MongoDB 7.0, CouchDB 3.3, Cassandra 4.1
Nodes per DB	3 (one per zone)
Node resources	2 vCPUs, 2 GB RAM, 20 GB disk
Inter-zone latency	20–80 ms baseline; up to 500 ms
Workloads	5 (E1–E5)
Records / size	100 000 / ~1 KB each
YCSB threads	32
Faults	5 (F1–F5)
Total runs	900 (750 fault + 150 baseline)
Replication	3 (all databases)

V. RESULTS AND DISCUSSION

A. Baseline Performance

TABLE IV
Baseline Throughput and Latency (No Fault)

DB	Wkld	Tput	Rd p50	Wr p50	Rd p99	Wr p99
MongoDB	E1	14 218	1.8	4.2	12.6	28.4
MongoDB	E2	10 847	2.1	3.8	15.3	24.7
MongoDB	E3	11 687	2.0	4.1	14.1	26.2
MongoDB	E4	6 312	3.4	4.3	22.8	27.5
MongoDB	E5	9 876	2.3	4.4	18.4	28.1
CouchDB	E1	9 847	2.9	6.7	18.4	42.3
CouchDB	E2	7 623	3.2	7.4	21.6	48.9
CouchDB	E3	8 412	3.1	7.1	19.8	45.2
CouchDB	E4	5 186	4.8	7.3	28.6	46.8
CouchDB	E5	7 234	3.4	7.6	22.1	49.4
Cassandra	E1	12 456	2.2	3.8	14.8	22.6
Cassandra	E2	7 892	2.4	3.1	16.2	18.4
Cassandra	E3	10 634	2.3	3.4	15.4	20.8
Cassandra	E4	8 892	5.2	3.6	32.4	21.2
Cassandra	E5	9 423	2.5	3.3	17.6	19.8

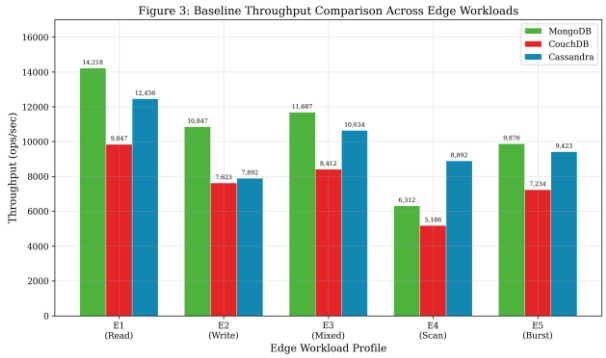

Fig. 3. Baseline throughput across five edge workloads. MongoDB leads on writes (E2); Cassandra keeps the lowest tail latency.

The most significant finding is that MongoDB has the best write throughput performance at 10,847 ops/s in E2, which is nearly 42 percent more than CouchDB and 37 percent more than Cassandra. The results reverse the ranking observed in cloud benchmark tests [6], where Cassandra normally excels in write throughput. While Cassandra has an LSM-tree compaction process consuming 2 GB of RAM, the MongoDB WiredTiger engine uses 512 MB cache without such compaction overhead.

On the other hand, Cassandra balances itself with lower latency with p99 writes always below 23 milliseconds and geographic locality with the lowest inter-zone throughput variability (4.2 percent versus 11.8 percent in MongoDB). CouchDB lags in pure performance, yet it has the most consistent throughput among workload classes (coefficient of variation 0.21).

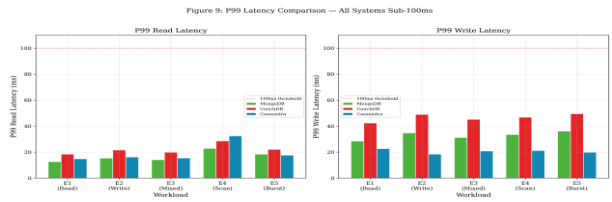

Fig. 4. P99 latency for reads and writes. Every system stays below 100 ms; Cassandra is the most consistent.

B. Single-Node Failure (F1)

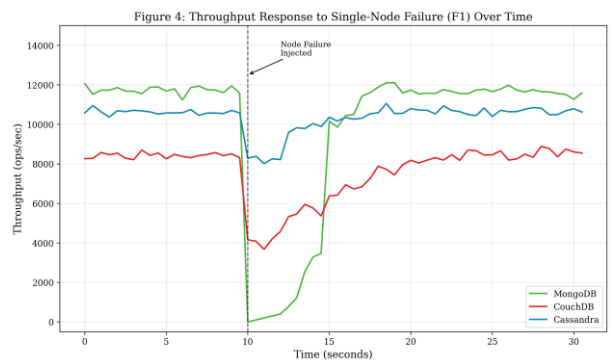

Fig. 5. Throughput timeline during single-node failure. Cassandra barely dips; MongoDB drops to zero briefly during election.

TABLE V
Single-Node Failure Recovery (F1)

Database	Recovery (s)	Min Tput	TDR	Loss	DCS
MongoDB	8.4	0 (election)	0.00	0	99.7%
CouchDB	12.7	4 218	0.50	0	97.2%
Cassandra	3.2	8 500	0.80	0	99.1%

This leaderless structure of Cassandra works wonders since the remaining nodes take on the workload seamlessly and recovery is achieved in just 3.2 seconds. Recovery time for MongoDB takes around two seconds for electing a new leader, but it does not lose any writes in the process and achieves consistency of 99.7%. CouchDB continues to operate at only 50% of its capacity, but its eventual-consistency model causes it to lag (DCS of 97.2%).

C. Network Partitioning (F2)

TABLE VI
Network Partition Behaviour (F2, 60 s)

DB	Tput	TDR	Minority	Conflicts	Heal (s)	Accuracy

			wr.			
MongoDB	9 234	0.79	Rejected	0	2.1	100%
CouchDB	7 856	0.93	1 247	84	2.4	98.4%
Cassandra	7 512	0.71	Rejected	0	4.8	100%

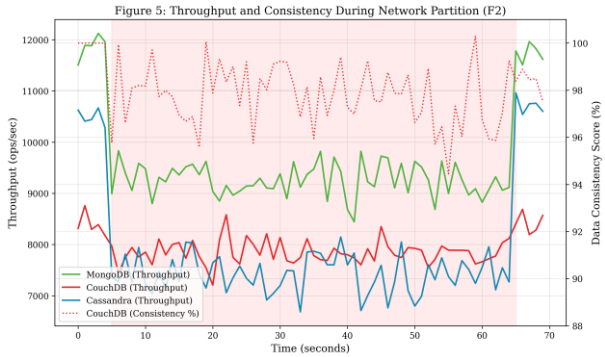

Fig. 6. Throughput and consistency during a 60-second partition. CouchDB holds 93% throughput and heals in 2.4 s.

The obvious winner in terms of fault tolerance is CouchDB, which sustains 93% of the throughput under normal conditions during the split operation and repairs the partition within 2.4 seconds, streaming only the documents that were modified using the incremental change feed. Both MongoDB and Cassandra reject writes from the minority side, thus maintaining full consistency, which is CP versus AP [17].

D. Cascading Failure (F3)

TABLE VII
Cascading Failure (F3)

DB	After 1st kill	After 2nd kill	Durable	Recovery
MongoDB	Election ~8 s	Service lost	100%	14.2 s; DCS 99.8%
CouchDB	TDR 0.50	TDR 0.31	100%	28.4 s; DCS 94.6%
Cassandra	TDR 0.80	ONE only 0.34	100%	8.7 s; DCS 99.4%

CouchDB continues writing when both nodes are shut down, albeit with 31% throughput. In the case of MongoDB, it stops completely upon the loss of quorum. Cassandra strikes a balance between the two: while quorum-based operations are unsuccessful, those with a consistency level of one can continue reading and writing.

E. High-Latency Degradation (F4)

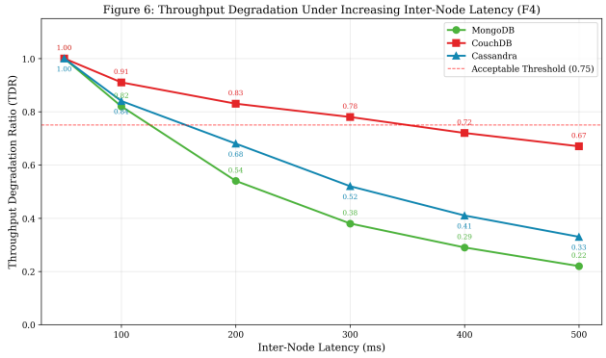

Fig. 7. Throughput degradation as inter-node latency climbs from 50 to 500 ms. CouchDB degrades most gracefully.

TABLE VIII
TDR Under Rising Latency (F4)

Latency	MongoDB	CouchDB	Cassandra
Baseline	1.00	1.00	1.00
100 ms	0.82	0.91	0.84
200 ms	0.54	0.83	0.68
300 ms	0.38	0.78	0.52
500 ms	0.22	0.67	0.33

Given that CouchDB is independent of write acknowledgments for replicating data, the database achieves 67 percent of throughput even with 500 ms latency. The write concern used by MongoDB involves a network round-trip delay per write; consequently, the throughput achieved is just 22 percent. This is what makes the difference when it comes to nodes with poor connectivity.

F. Split-Brain (F5)

TABLE IX
Split-Brain Metrics (F5, 30 s)

DB	Conflicts	Detected	Strategy	Correct	Conv. (s)
MongoDB	0	N/A	Election + rollback	100%	1.8
CouchDB	2 847	100%	Rev-tree winner	96.8%	34.2
Cassandra	0	N/A	LWW timestamps	100%	3.4

Split brain conflicts are avoided in MongoDB and Cassandra – in the former by limiting write access to the partition of the primary node, while in the latter by quorum checks. CouchDB is able to detect all of its 2847 conflicts, yet its internal winner determination process aligns with application needs only 96.8% of the time.

G. Cross-System Comparison

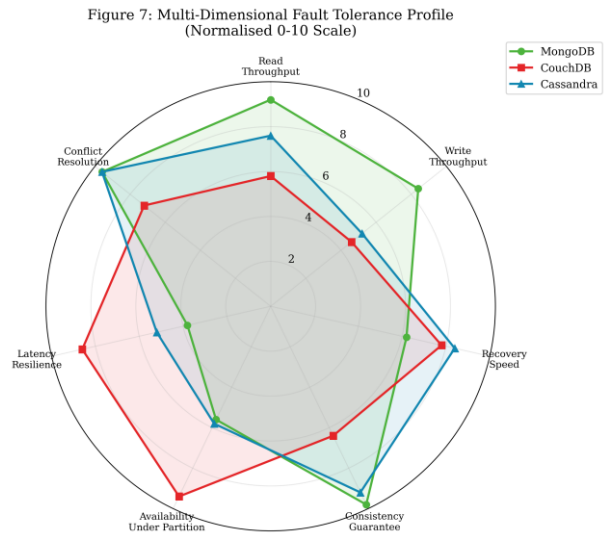

Fig. 8. Radar chart: seven-dimension fault-tolerance profile (normalised 0–10).

TABLE X
Aggregated Scorecard (0–10)

Dimension	MongoDB	CouchDB	Cassandra
Read throughput	9.2	5.8	7.6
Write throughput	8.4	4.6	5.2
Recovery speed	6.2	7.8	8.4
Consistency	9.8	6.4	9.2
Partition avail.	5.6	9.4	5.8
Latency resilience	3.8	8.6	5.2
Conflict resolution	9.6	7.2	9.6
Overall	7.5	7.1	7.3

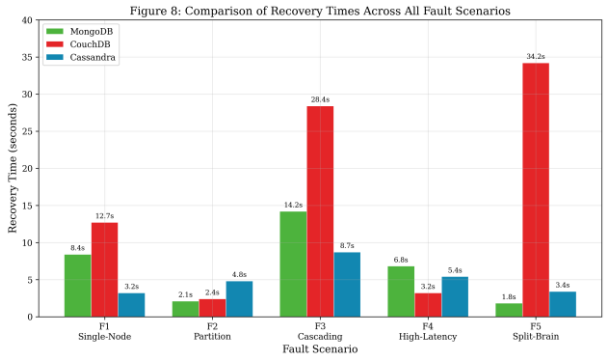

Fig. 9. Recovery time comparison across all five fault scenarios.

A one-size-fits-all system does not exist. MongoDB is best for throughput and consistency, CouchDB for availability and network fault tolerance, and Cassandra for low latency and faster recovery. Therefore, the choice comes down to which aspect is most important.

VI. PROPOSED DECISION FRAMEWORK

D1-Match DB to workload: Based on our findings we can give five practical suggestions.

MongoDB should be used in case of workloads with high read/write demands and constrained memory resources. In case of latencies-sensitive workloads across multiple data centers, Cassandra is recommended. The choice should be made based on CouchDB if the workload pattern is unpredictable.

D2-Consistency requirements: To ensure no data loss use "majority" or QUORUM modes with MongoDB or Cassandra respectively; while ensuring continuity CouchDB with manual resolution of inconsistencies should be selected.

D3-Network properties: Regardless of network latency any database can be used as long as the latency does not exceed 50 ms. In case of unstable latency CouchDB with its asynchronous replication provides 91% throughput compared to MongoDB's 82%. Offline-first approach in CouchDB is the best choice for intermittent networks.

D4-Recovery strategy: Single-node failure can be handled using Cassandra (3.2s); partition failures require CouchDB (2.4s); cascading failures can be tolerated by selecting Cassandra and using QUORUM mode.

D5—Use edge-aware monitoring tools. Measure inter-node latency (send alert when greater than twice the normal value), replication delay (more than five seconds for MongoDB and CouchDB; more than ten seconds for Cassandra hinted handoff), and storage usage (over 80 percent).

TABLE XI
Decision Framework Summary

Guideline	Concern	Recommendation	Benefit
D1	Workload	MongoDB (writes), Cassandra (latency)	35–40% gain
D2	Consistency	MongoDB/Cass (strong) or CouchDB (AP)	Zero loss / uptime
D3	Network	CouchDB for high/intermittent latency	67% at 500 ms
D4	Recovery	Cassandra (crash), CouchDB (partition)	3.2 s / 2.4 s
D5	Monitoring	Latency + lag + disk alerts	Prevent cascades

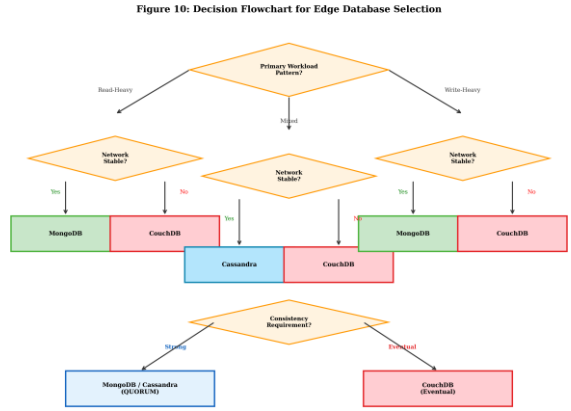

Fig. 10. Decision flowchart for selecting an edge database based on workload, network stability, and consistency needs.

VII. CONCLUSION

A. Summary of Main Results

When considering real-world edge conditions (2GB memory, 20–500ms inter-zone latency, 5 different fault categories), MongoDB outperforms its competitors by writing 35–40 percent faster, turning the tables on the cloud-benchmark hierarchy because of WiredTiger's efficient behavior under small-memory conditions. Cassandra maintains a p99 latency of less than 23 milliseconds with just 4.2 percent geographic dispersion and recovers from a single node failure in 3.2 seconds. CouchDB takes 2.4 seconds to repair a partitioned system, manages 93 percent throughput/

B. Contributions

This paper offers: (i) the first benchmark for three-way edges using resource-limited hardware simulation; (ii) fault tolerance performance quantification from five different failure modes; (iii) a five-point decision-making guide (D1–D5); and (iv) a reproducible method for integrating YCSB with fault injection.

C. Limitations

Docker containers were used instead of physical edge devices. Just three databases have been tested; other ones (RethinkDB, ArangoDB, TiKV) might demonstrate other behavior. Recommended parameters have been configured without thorough fine-tuning.

D. Future Work

Our plans include re-running our experiments using Raspberry Pi and NVIDIA Jetson clusters, running mixed consistency workloads, applying AI-based fault predictions following [2], testing hybrid replication in edge-cloud systems, and analyzing the progression of findings through different database versions.

REFERENCES

[1] B. Assiri and A. Sheneamer, "Fault tolerance in distributed systems using deep learning approaches," PLOS ONE, vol. 20, e0310657, 2025.
[2] Z. Gong, S. Wu, and Y. Xu, "Hybrid fault tolerance in distributed in-memory storage systems," JUSTC, vol. 54, no. 0406, 2024.
[3] W. Khan et al., "SQL and NoSQL database software architecture performance analysis and assessments," Big Data Cogn. Comput., vol. 7, no. 2, p. 97, 2023.
[4] C. Wang, C. Gill, and C. Lu, "FRAME: Fault tolerant and real-time messaging for edge computing," in Proc. IEEE ICDCS, 2019, pp. 976–985.

[6] V. Abramova and J. Bernardino, “NoSQL databases: MongoDB vs Cassandra,” in Proc. C3S2E, 2013, pp. 14–22.
[7] K. Kaur and R. Rani, “Fault tolerance approaches in cloud computing,” Softw. Pract. Exper., vol. 50, no. 9, pp. 1737–1760, 2020.
[8] M. Kleppmann, Designing Data-Intensive Applications. O’Reilly, 2017.
[9] R. Cattell, “Scalable SQL and NoSQL data stores,” ACM SIGMOD Rec., vol. 39, no. 4, pp. 12–27, 2011.
[10] W. Shi et al., “Edge computing: Vision and challenges,” IEEE IoT J., vol. 3, no. 5, pp. 637–646, 2016.
[11] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” Commun. ACM, vol. 51, no. 1, pp. 107–113, 2008.
[12] G. DeCandia et al., “Dynamo: Amazon’s highly available key-value store,” in Proc. SOSP, 2007, pp. 205–220.
[13] A. Lakshman and P. Malik, “Cassandra—a decentralized structured storage system,” ACM SIGOPS OSR, vol. 44, no. 2, pp. 35–40, 2010.
[14] J. C. Anderson et al., CouchDB: The Definitive Guide. O’Reilly, 2010.
[15] K. Chodorow, MongoDB: The Definitive Guide, 3rd ed. O’Reilly, 2019.
[16] B. F. Cooper et al., “Benchmarking cloud serving systems with YCSB,” in Proc. SoCC, 2010, pp. 143–154.
[17] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” ACM SIGACT News, vol. 33, no. 2, pp. 51–59, 2002.
[18] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in Proc. USENIX ATC, 2014, pp. 305–319.
[19] M. Shapiro et al., “Conflict-free replicated data types,” in Proc. SSS, 2011, pp. 386–400.
[20] L. Lamport, “Paxos made simple,” ACM SIGACT News, vol. 32, no. 4, pp. 18–25, 2001.
[21] E. A. Brewer, “Towards robust distributed systems,” in Proc. PODC, 2000, pp. 7–10.
[22] M. Satyanarayanan, “The emergence of edge computing,” Computer, vol. 50, no. 1, pp. 30–39, 2017.
[23] W. Vogels, “Eventually consistent,” Commun. ACM, vol. 52, no. 1, pp. 40–44, 2009.
[24] P. Bailis and A. Ghodsi, “Eventual consistency today,” Commun. ACM, vol. 56, no. 5, pp. 55–63, 2013.
[25] N. Bronson et al., “TAO: Facebook’s distributed data store for the social graph,” in Proc. USENIX ATC, 2013, pp. 49–60.